

1. Hàm `insert(int father, int data)`

- **Độ phức tạp trung bình:** $O(n)$ (với n là số node trong cây), vì việc tìm kiếm node cha (`findNode`) có thể duyệt qua toàn bộ cây.
- **Trường hợp tốt nhất:** $O(1)$ khi node cha là root hoặc khi cây có cấu trúc thuận lợi, node cha được tìm thấy ngay ở cấp trên.
- **Trường hợp xấu nhất:** $O(n)$, nếu cần duyệt qua toàn bộ cây để tìm node cha.

2. Hàm `findNode(Node* node, int data)`

- **Độ phức tạp trung bình:** $O(n)$, với n là số lượng node trong cây, vì hàm này phải duyệt toàn bộ cây trong trường hợp xấu nhất.
- **Trường hợp tốt nhất:** $O(1)$, nếu tìm thấy node ngay ở đầu cây (node gốc).
- **Trường hợp xấu nhất:** $O(n)$, khi cần duyệt qua toàn bộ cây mà node không tồn tại trong cây.

3. Hàm `getNumOfChild(Node* node)`

- **Độ phức tạp trung bình:** $O(n)$, với n là số lượng node trong cây con bắt đầu từ node đầu vào, vì phải duyệt qua tất cả node con.
- **Trường hợp tốt nhất:** $O(1)$, nếu node không có con (số lượng con là 0).
- **Trường hợp xấu nhất:** $O(n)$, khi node có rất nhiều con và phải duyệt qua tất cả các con của nó.

4. Hàm `remove(int data)`

- **Độ phức tạp trung bình:** $O(n)$, bao gồm việc tìm kiếm node cần xoá và xoá nó cùng với các con của nó.
- **Trường hợp tốt nhất:** $O(1)$, khi node cần xoá là node gốc không có con.
- **Trường hợp xấu nhất:** $O(n)$, khi node cần xoá nằm sâu trong cây và có nhiều con.

5. Hàm `preorder(Node* node)`

- **Độ phức tạp trung bình:** $O(n)$, vì phải duyệt qua toàn bộ cây trong trường hợp tổng quát.
- **Trường hợp tốt nhất:** $O(1)$, khi cây chỉ có một node.
- **Trường hợp xấu nhất:** $O(n)$, khi cây có cấu trúc phức tạp với nhiều node cần duyệt.

6. Hàm `postorder(Node* node)`

- **Độ phức tạp trung bình:** $O(n)$, tương tự như `preorder`, phải duyệt qua toàn bộ cây.
- **Trường hợp tốt nhất:** $O(1)$, khi chỉ có một node.
- **Trường hợp xấu nhất:** $O(n)$, khi cây có cấu trúc phức tạp và cần duyệt qua tất cả các node.

7. Hàm **isBinaryTree(Node* node)**

- **Độ phức tạp trung bình:** $O(n)$, cần duyệt qua toàn bộ cây để đếm số lượng con của mỗi node.
- **Trường hợp tốt nhất:** $O(1)$, nếu node không có con hoặc chỉ có hai con.
- **Trường hợp xấu nhất:** $O(n)$, khi có node có hơn hai con và phải kiểm tra toàn bộ cây.

8. Hàm **isBinarySearchTree(Node* node)**

- **Độ phức tạp trung bình:** $O(n)$, tương tự như **isBinaryTree**, vì cần kiểm tra tính chất của toàn bộ cây.
- **Trường hợp tốt nhất:** $O(1)$, nếu node gốc không có con.
- **Trường hợp xấu nhất:** $O(n)$, khi cây có nhiều node cần kiểm tra.

9. Hàm **isMaxHeapTree(Node* node)**

- **Độ phức tạp trung bình:** $O(n)$, phải duyệt qua toàn bộ cây để kiểm tra tính chất Max-Heap.
- **Trường hợp tốt nhất:** $O(1)$, nếu node gốc không có con hoặc thỏa mãn Max-Heap ngay từ đầu.
- **Trường hợp xấu nhất:** $O(n)$, nếu có nhiều node không thỏa mãn tính chất Max-Heap và phải duyệt qua tất cả.

10. Hàm **inorder(Node* node)**

- **Độ phức tạp trung bình:** $O(n)$, vì đây là phép duyệt qua toàn bộ cây (giả định cây nhị phân).
- **Trường hợp tốt nhất:** $O(1)$, nếu cây chỉ có một node.
- **Trường hợp xấu nhất:** $O(n)$, khi cây đầy đủ và cần duyệt qua tất cả các node.

11. Hàm **height(Node* node)**

- **Độ phức tạp trung bình:** $O(n)$, vì cần duyệt qua tất cả các node để tính độ cao.
- **Trường hợp tốt nhất:** $O(1)$, nếu cây chỉ có một node (hoặc không có con).
- **Trường hợp xấu nhất:** $O(n)$, khi cây không cân đối và có chiều cao lớn nhất.

12. Hàm **depth(int data)**

- **Độ phức tạp trung bình:** $O(n)$, cần duyệt qua cây để tìm node.
- **Trường hợp tốt nhất:** $O(1)$, nếu node cần tìm là gốc.
- **Trường hợp xấu nhất:** $O(n)$, khi node nằm ở độ sâu lớn nhất trong cây.

13. Hàm **numOfLeaves(Node* node)**

- **Độ phức tạp trung bình:** $O(n)$, phải duyệt qua toàn bộ cây để đếm số lá.

- Trường hợp tốt nhất: $O(1)$, nếu cây chỉ có một node lá.
- Trường hợp xấu nhất: $O(n)$, khi cây có nhiều node lá và cần kiểm tra toàn bộ cây.

14. Hàm **findMax(Node* root)**

- Độ phức tạp trung bình: $O(n)$, phải duyệt qua toàn bộ cây để tìm giá trị lớn nhất.
- Trường hợp tốt nhất: $O(1)$, nếu node gốc có giá trị lớn nhất.
- Trường hợp xấu nhất: $O(n)$, khi node có giá trị lớn nhất nằm sâu nhất trong cây.

15. Hàm **findMaxChild(Node* root)**

- Độ phức tạp trung bình: $O(n)$, phải duyệt qua toàn bộ cây để tìm node có nhiều con nhất.
- Trường hợp tốt nhất: $O(1)$, nếu node gốc có nhiều con nhất.
- Trường hợp xấu nhất: $O(n)$, khi node có nhiều con nhất nằm sâu trong cây.